

Programming Assignment 1

February 23, 2026

0.0.1 Objectives:

- To plot the empirical and the theoretical distribution of the overall data without regard to classes.
- To plot the empirical and the theoretical distribution of the data of each class.
- To study the joint behavior of two random variables by plotting a 2D scatter plot and a contour plot of the 2D Gaussian distribution.
- To build a threshold classifier on one feature (one random variable), perform pattern recognition, and evaluate the performance of the classifier.
- To build a threshold classifier on two features (two random variables), perform pattern recognition, and evaluate the performance of the classifier.

In this assignment, we will look at how python can be used to analyze data using the pandas data analysis package, and how we can model data using probability distributions studied in class.

We will be using a data file which has been extracted from the Iris Flower Data Set, which is perhaps the best known database to be found in the pattern recognition literature. The original dataset consists of 50 samples from each of three species of Iris (setosa, versicolor and virginica). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters. For more information, please see the following page at the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/iris>

In this problem, we will be working with a smaller dataset, which considers only two types of irises: versicolor and virginica, and two random variable: petal length and width.

Download the file “iris data.xlsx” from Canvas to complete the homework.

In **Section 1**, you will be working with one random variable, i.e., petal length.

In **Section 2**, you will be working with two joint random variables, i.e., petal length and width.

In **Section 3**, you will look into pattern classification with petal length and width.

We recommend you use the pandas package, the numpy package and the scipy and matplotlib.pyplot libraries in python, as you have done in previous homeworks.

After you have completed the notebook, export it as pdf for submission. You can do this in one of two ways: 1. Go to File, click Save and Export Notebook as, click PDF. 2. Go to File, click Save and Export Notebook as, click HTML (.html), then convert the html file to pdf file.

We first load the datafile using the **pandas.read_excel** function into a **pandas.DataFrame**.

```
[17]: # load pandas, a data analysis package
# load numpy, a scientific computing package
# load scipy.stats, a module contains probability functions
# load matplotlib.pyplot, a framework provides a Matlab-like plotting
import pandas as pd
import numpy as np
import scipy.stats
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.stats import multivariate_normal

data = pd.read_excel('iris data.xlsx', index_col = 'Index')
```

There are 100 entries in total in this data set. The first 50 entries contains measurements of petal length and width from irises in the **versicolor** class. We can see a few examples using the **pandas.DataFrame.head** function, which returns the **first** few entries in the data frame.

```
[18]: data.head()
```

```
[18]:
```

	Petal length	Petal width	Class
Index			
1	4.7	1.4	Iris-versicolor
2	4.5	1.5	Iris-versicolor
3	4.9	1.5	Iris-versicolor
4	4.0	1.3	Iris-versicolor
5	4.6	1.5	Iris-versicolor

The last 50 entries contain measurements of the petal length and width from irises in the **virginica** class. We can see a few examples using the **pandas.DataFrame.tail** function, which returns the **last** few entries in the data frame.

```
[19]: data.tail()
```

```
[19]:
```

	Petal length	Petal width	Class
Index			
96	5.2	2.3	Iris-virginica
97	5.0	1.9	Iris-virginica
98	5.2	2.0	Iris-virginica
99	5.4	2.3	Iris-virginica
100	5.1	1.8	Iris-virginica

From these lines, you should get the general idea that the petals of virginica irises are generally longer than the petals of versicolor irises. Thus, if you observe an iris with longer petals, it is “more likely” to be a virginica than a versicolor. Similarly, if you observe an iris with shorter petals, it is “more likely” to be a versicolor. This assignment makes this intuition more precise using probability theory.

Section 1: Working with one random variable, i.e., petal length.

Part a: Plot the normalized histogram of the petal length for all the data without regard to which

species from 2 to 7 with bin size 0.1. The normalized histogram is the count in each bin divided by the number of observations and divided by the bin width. In python, this can be selected by passing “density=True” into the `matplotlib.pyplot.hist` or `pandas.DataFrame.hist` functions.

```
[20]: # Part a: normalized histogram of petal length (all species), from 2 to 7,
      ↪ binsize 0.1

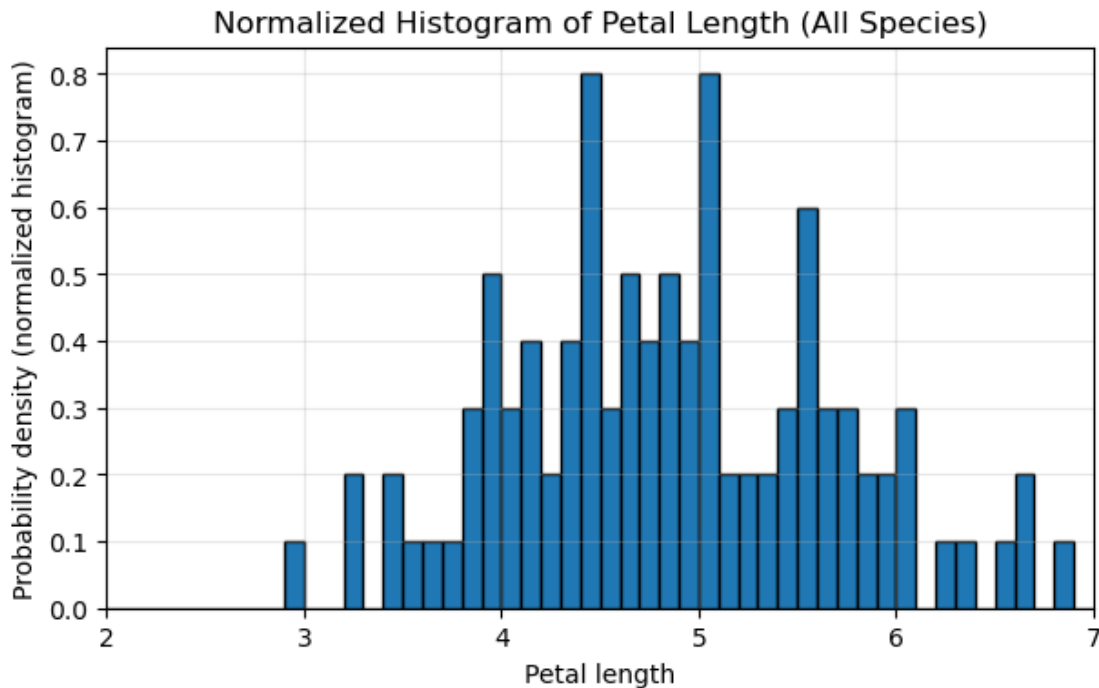
x = data["Petal length"].dropna()

bin_width = 0.1
bins = np.arange(2.0, 7.0 + bin_width + 1e-12, bin_width)

plt.figure(figsize = (7, 4))
plt.hist(x, bins = bins, density = True, edgecolor = "black")
plt.xlim(2,7)

plt.xlabel("Petal length")
plt.ylabel("Probability density (normalized histogram)")
plt.title("Normalized Histogram of Petal Length (All Species)")
plt.grid(alpha = 0.3)
plt.show
```

```
[20]: <function matplotlib.pyplot.show(close=None, block=None)>
```



Part b: Calculate empirical mean m and standard deviation σ of the petal length over all of the

data without regard to species. For a set of data $\{r_1, r_2, \dots, r_n\}$, the empirical mean is given by

$$m = \frac{1}{n} \sum_{i=1}^n r_i$$

and the empirical standard deviation is given by

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (r_i - m)^2}$$

If you have read the data in using the pandas package, then you can compute the mean and standard deviation easily using the `pandas.DataFrame.mean` and `pandas.DataFrame.std` functions.

```
[21]: # Part b: Empirical mean m and empirical standard deviation sigma of petal
      ↪ length (all species)

x = data["Petal length"].dropna()

m = x.mean()
sigma = x.std(ddof = 1)

print(f"Empirical mean m = {m:.6f}")
print(f"Empirical std \u03C3 = {sigma:.6f}")
```

Empirical mean m = 4.906000

Empirical std = 0.825578

Part c: Assume that the probability density function of the petal length, $f_X(x)$, is given by a Gaussian distribution with mean and standard deviation computed in **Part b**. You can compute values of the Gaussian distribution in python using the `scipy.stats.norm.pdf` function. Note that the Gaussian distribution is also called the normal distribution.

Compare the normalized histogram in **Part a** with the plot of $f_X(x)$ by plotting both together in the same figure for x from 2 to 7. Does this look like a good model of the data?

```
[22]: # Part c: overlay normalized histogram (Part a) with Gaussian pdf using m,
      ↪ sigma from Part b
from scipy.stats import norm

x = data["Petal length"].dropna()

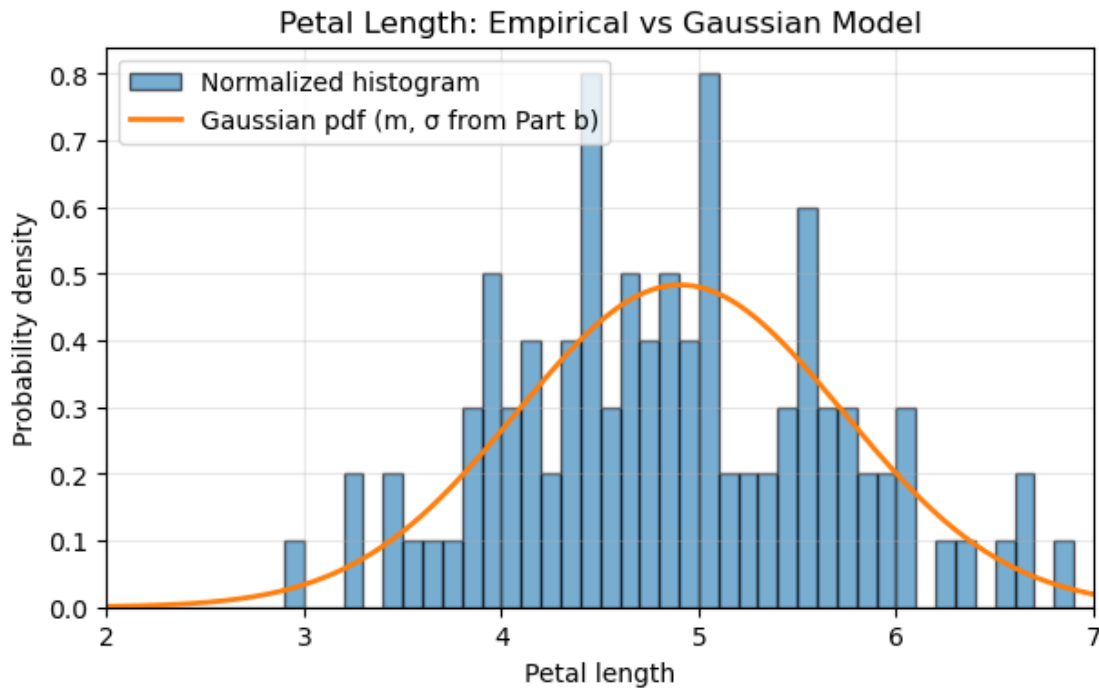
m = float(x.mean())
sigma = float(x.std(ddof=1))

x_grid = np.linspace(2.0, 7.0, 600)
pdf = norm.pdf(x_grid, loc=m, scale=sigma)

plt.figure(figsize=(7, 4))
plt.hist(x, bins=bins, density=True, edgecolor="black", alpha=0.6,
      ↪ label="Normalized histogram")
```

```
plt.plot(x_grid, pdf, linewidth=2, label="Gaussian pdf (m, \u03C3 from Part b)")
plt.xlim(2, 7)

plt.xlabel("Petal length")
plt.ylabel("Probability density")
plt.title("Petal Length: Empirical vs Gaussian Model")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```



Part c Answer:

It's not a very good model. The normalized histogram of petal length over **all species combined** is not well-captured by a single Gaussian: the data show noticeable asymmetry and have multiple clusters (due to the mixing of different species), while a Gaussian is unimodal and symmetric; a mixture model by species would fit better.

Part d: Now generate plots of normalized histograms of the petal length for each species separately. You can separate data into different groups using the **pandas.DataFrame.groupby** command. Then for each group, you can use the **pandas.DataFrame.hist** command to plot the histogram.

[23]: *# Part d: normalized histogram of petal length for each species (Class) separately*

```
bin_width = 0.1
bins = np.arange(2.0, 7.0 + bin_width + 1e-12, bin_width)
```

```

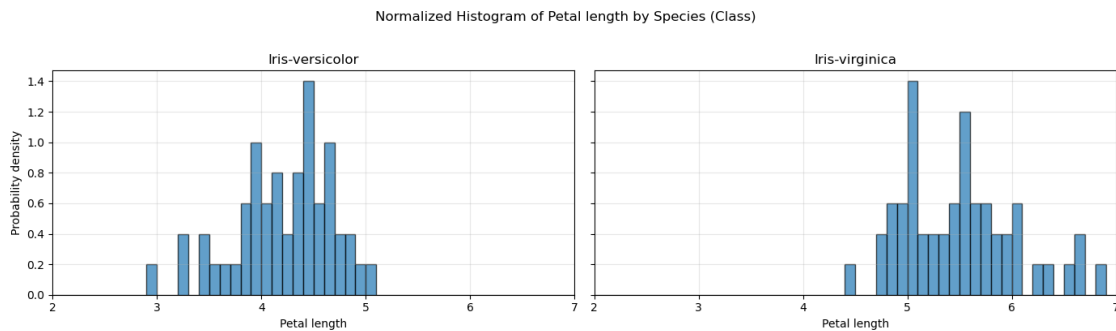
groups = data.groupby("Class")

fig, axes = plt.subplots(1, len(groups), figsize = (14,4), sharey = True)
if len(groups) == 1:
    ax = axes

for ax, (cls, g) in zip(axes, groups):
    g["Petal length"].dropna().plot.hist(
        bins = bins,
        density = True,
        edgecolor = "black",
        alpha = 0.7,
        ax = ax
    )
    ax.set_xlim(2, 7)
    ax.set_title(cls)
    ax.set_xlabel("Petal length")
    ax.grid(alpha = 0.3)

axes[0].set_ylabel("Probability density")
fig.suptitle("Normalized Histogram of Petal length by Species (Class)", y = 1.02)
plt.tight_layout()
plt.show()

```



Part e: Use your knowledge of probability to model probability distributions of the petal lengths given virginica and versicolor, $f_{X|species}(x|virginica)$ and $f_{X|species}(x|versicolor)$. For each species, compare the normalized histogram with the conditional density by plotting them both in the same figure for x from 2 to 7.

```

[24]: # Part e: model conditional densities  $f_{X|species}(x|versicolor)$  and  $f_{X|species}(x|virginica)$ 
      # using Gaussian with species-specific empirical mean/std, and overlay with normalized histograms.

```

```

# 0) import
from scipy.stats import norm

bin_width = 0.1
bins = np.arange(2.0, 7.0 + bin_width + 1e-12, bin_width)
x_grid = np.linspace(2.0, 7.0, 600)

target_species = ["Iris-versicolor", "Iris-virginica"]

fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)

for ax, sp in zip(axes, target_species):
    # 1) Filter to species
    x_sp = data.loc[data["Class"] == sp, "Petal length"].dropna()

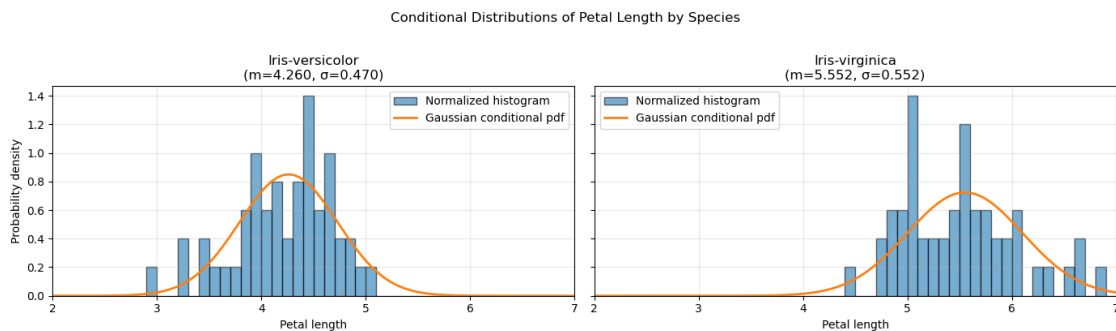
    # 2) Empirical params for this species (scalar!)
    m_sp = float(x_sp.mean())
    sigma_sp = float(x_sp.std(ddof=1))

    # 3) Conditional density model: Gaussian  $N(m_{sp}, \sigma_{sp}^2)$ 
    pdf_sp = norm.pdf(x_grid, loc=m_sp, scale=sigma_sp)

    # 4) Plot: normalized histogram + pdf
    ax.hist(x_sp, bins=bins, density=True, edgecolor="black", alpha=0.6,
    label="Normalized histogram")
    ax.plot(x_grid, pdf_sp, linewidth=2, label="Gaussian conditional pdf")
    ax.set_xlim(2, 7)
    ax.set_title(f"{sp}\n(m={m_sp:.3f}, \u03C3={sigma_sp:.3f})")
    ax.set_xlabel("Petal length")
    ax.grid(alpha=0.3)
    ax.legend()

axes[0].set_ylabel("Probability density")
fig.suptitle("Conditional Distributions of Petal Length by Species", y=1.02)
plt.tight_layout()
plt.show()

```



Part f: Use the total probability theorem to combine the two conditional densities $f_{X|species}(x|virginica)$ and $f_{X|species}(x|versicolor)$ to obtain a new model distribution for the petal length, $g_X(x)$. Compare the plot of $g_X(x)$ with the normalized histogram in **Part a** by plotting both in the same figure for x from 2 to 7. How does this compare with the single Gaussian assumption in part **Part c**?

```
[25]: # Part f
from scipy.stats import norm

# 0) variables settings
bin_width = 0.1
bins = np.arange(2.0, 7.0 + bin_width + 1e-12, bin_width)
x_grid = np.linspace(2.0, 7.0, 600)

# 1) Data used in a histogram over [2,7]
x_all = data["Petal length"].dropna()
x_all_in_range = x_all[(x_all >= 2.0) & (x_all) <= 7.0]

# 2) Extract the two species samples (within [2,7] to match the plotted
    ↪ histogram target)
sp_v = "Iris-virginica"
sp_vs = "Iris-versicolor"

x_v = data.loc[data["Class"] == sp_v, "Petal length"].dropna()
x_vs = data.loc[data["Class"] == sp_vs, "Petal length"].dropna()

## constrain x_v and x_vs in the range of [2,7]
x_v = x_v[(x_v >= 2.0) & (x_v <= 7.0)]
x_vs = x_vs[(x_vs >= 2.0) & (x_vs <= 7.0)]

# 3) Conditional destination: Gaussian per species using emperical (m_s,
    ↪ sigma_s)
m_v, s_v = float(x_v.mean()), float(x_v.std(ddof = 1))
m_vs, s_vs = float(x_vs.mean()), float(x_vs.std(ddof = 1))

f_v = norm.pdf(x_grid, loc = m_v, scale = s_v)      # f_{X|species}(x/virginica)
f_vs = norm.pdf(x_grid, loc = m_vs, scale = s_vs)   #
    ↪ f_{X|species}(x/versicolor)

# 4) Total probability theorem: g_X(x) = P(v)*f_v + P(vs)*f_vs
# since our histogram only counts x in [2,7], and setosa is essentially outside
    ↪ this range,
# we estimate weights using the *in-range* sample proportions for best
    ↪ apples-to-apples comparison.
```

```

count_v = len(x_v)
count_vs = len(x_vs)
w_v = count_v / (count_v + count_vs)
w_vs = count_vs / (count_v + count_vs)

g = w_v * f_v + w_vs * f_vs      # new distribution

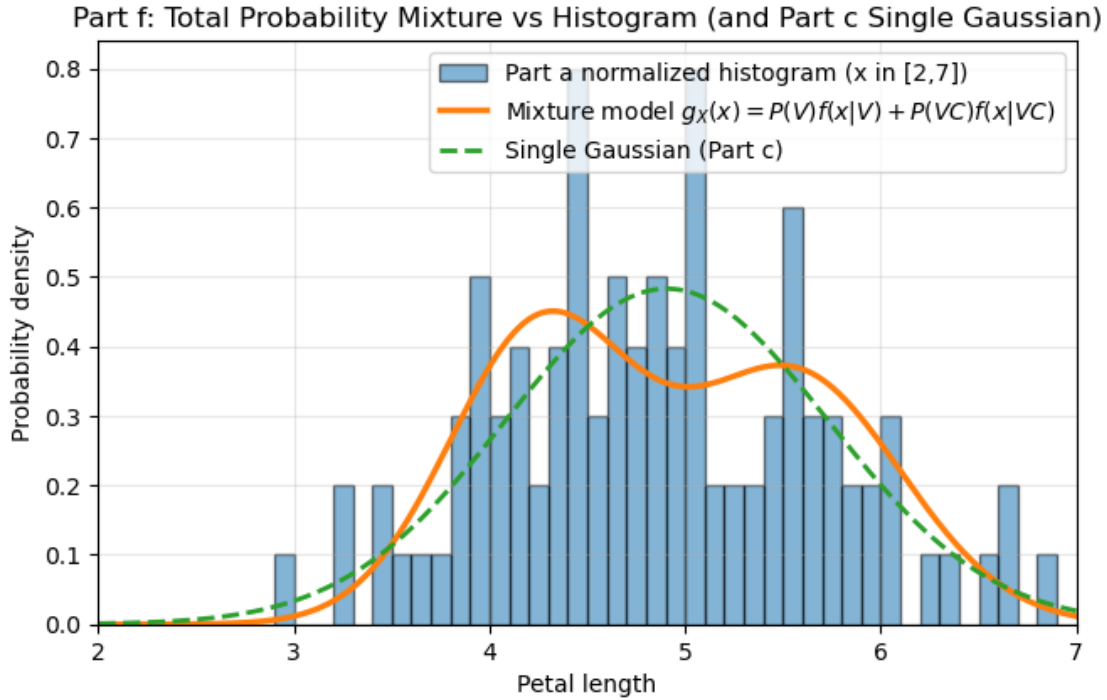
# 5) Compare with Part c single Gaussian
m_single = float(x_all_in_range.mean())
s_single = float(x_all_in_range.std(ddof = 1))
single_pdf = norm.pdf(x_grid, loc = m_single, scale = s_single)

# 6) Plot: histogram vs g_X, and single Gaussian for comparison
plt.figure(figsize = (7.5, 4.5))
plt.hist(x_all_in_range, bins = bins, density = True, edgecolor = "black",
        ↪alpha = 0.55, label = "Part a normalized histogram (x in [2,7])")
plt.plot(x_grid, g, linewidth = 2.5, label = r"Mixture model ↪
        ↪$g_X(x)=P(V)f(x|V)+P(V_C)f(x|V_C)$")
plt.plot(x_grid, single_pdf, linewidth = 2, linestyle = "--", label = "Single ↪
        ↪Gaussian (Part c)")

plt.xlim(2,7)
plt.xlabel("Petal length")
plt.ylabel("Probability density")
plt.title("Part f: Total Probability Mixture vs Histogram (and Part c Single ↪
        ↪Gaussian)")
plt.grid(alpha = 0.3)
plt.legend()
plt.show()

print(f"Weight in [2,7]: P(virginica) = {w_v:.3f}, P(versicolor) = {w_vs:.3f}")
print(f"Virginica:      m = {m_v:.4f}, \u03C3 = {s_v:.4f}")
print(f"Versicolor:    m = {m_vs:.4f}, \u03C3 = {s_vs:.4f}")
print(f"Single Gauss:   m = {m_single:.4f}, \u03C3 = {s_single:.4f}")

```



Weight in $[2, 7]$: $P(\text{virginica}) = 0.500$, $P(\text{versicolor}) = 0.500$
 Virginica: $m = 5.5520$, $= 0.5519$
 Versicolor: $m = 4.2600$, $= 0.4699$
 Single Gauss: $m = 4.9060$, $= 0.8256$

Part f Answer: Using the total probability theorem, the new model ($g_X(x) = P(V)f(x|V) + P(VC)f(x|VC)$) is a **mixture of two Gaussians** (one per species). Compared with the normalized histogram from Part a (over $(x \in [2, 7])$), the mixture model typically matches the data better than the single Gaussian model in Part c, because combining two species introduces **multi-modality / asymmetry** that a single Gaussian (unimodal and symmetric) cannot capture. In other words, the mixture model provides a more flexible and realistic fit for the aggregated distribution.

Section 2: Working with two joint random variables, i.e., petal length and width.

Below, we visualize the joint behavior of two random variables by plotting a 2D scatter plot of petal length and width.

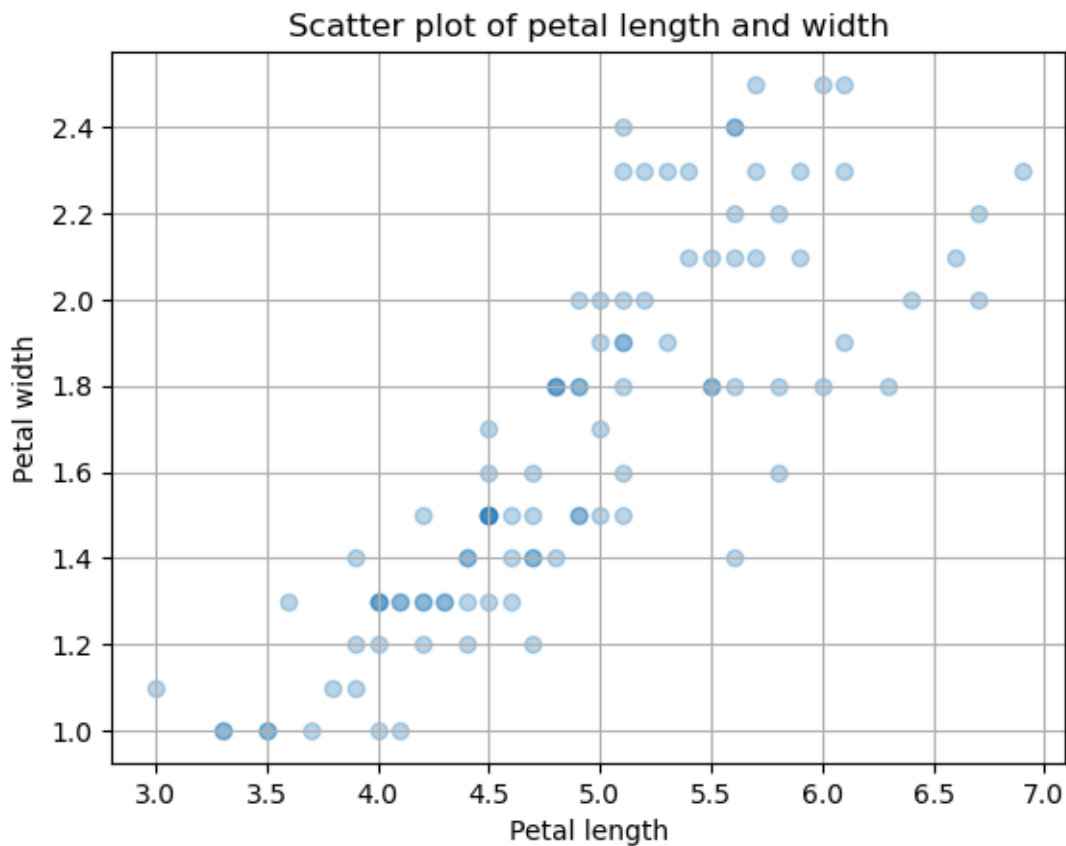
We plot the 2D scatter plots using `ax.scatter`.

```
[26]: # create figure
fig, ax = plt.subplots()

# create scatter plot
ax.scatter(data['Petal length'], data['Petal width'], alpha=0.3)
```

```
axlim = [2, 7, 0, 3]
plt.xlabel('Petal length')
plt.ylabel('Petal width')
plt.grid(True)
ax.set_title('Scatter plot of petal length and width')

plt.show()
```



We start by modelling the joint distribution of petal length and width using a 2D Gaussian distribution $f_{X,Y}(x,y)$ with mean vector m and covariance matrix C .

The code below estimates the mean and covariance that best fit the data using the **pandas.DataFrame.mean** and **pandas.DataFrame.cov** functions.

```
[27]: # Compute the mean vector and covariance matrix
m = data[['Petal length', 'Petal width']].mean()
C = data[['Petal length', 'Petal width']].cov()
rho = data[['Petal length', 'Petal width']].corr(method = 'pearson')

print('The mean vector of petal length and width is\n')
print(m)
```

```

print('\nThe covariance matrix of petal length and width is\n')
print(C)

print('\nThe correlation coefficient matrix of petal length and width is\n')
print(rho)

```

The mean vector of petal length and width is

```

Petal length    4.906
Petal width     1.676
dtype: float64

```

The covariance matrix of petal length and width is

	Petal length	Petal width
Petal length	0.681580	0.288731
Petal width	0.288731	0.180428

The correlation coefficient matrix of petal length and width is

	Petal length	Petal width
Petal length	1.000000	0.823348
Petal width	0.823348	1.000000

The code below overlays a contour plot of the 2D Gaussian that best fits the data onto the scatterplot.

```

[28]: # create a multivariate Gaussian random variable with mean m and covariance
      ↪matrix C
dist = multivariate_normal(m, C)

# create limits of plot
axlim = [2, 7, 0, 3]

# create set of (x,y) positions to compute the pdf at
x, y = np.meshgrid( np.linspace(axlim[0], axlim[1]), np.linspace(axlim[2],
      ↪axlim[3]) )
pos = np.dstack((x, y))

# compute the joint Gaussian pdf
f_XY = dist.pdf(pos)

# create the plot
fig, ax = plt.subplots()

# scatter plot
ax.scatter(data['Petal length'], data['Petal width'], alpha=0.3)

```

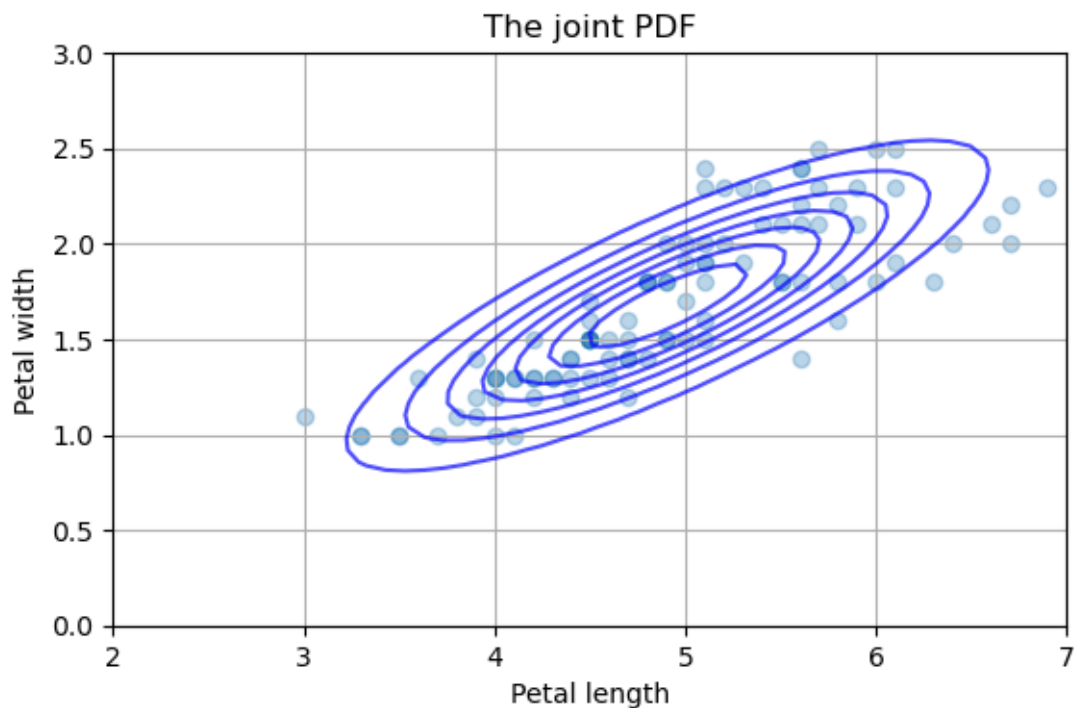
```

# pdf contour plot
ax.contour(x, y, f_XY, colors='b', alpha = 0.7)

# set axes and labels
ax.axis('scaled')
ax.axis(axlim)
plt.xlabel('Petal length')
plt.ylabel('Petal width')
plt.grid(True)
ax.set_title('The joint PDF')

plt.show()

```



Now, we generate the scatter plot of petal length and width for each species separately. We separate data into different groups using the **pandas.DataFrame.groupby** command.

```

[29]: # create figure
fig, ax = plt.subplots()

# create one scatter plot for each flower type
for name, groups in data.groupby('Class'):
    ax.scatter(x='Petal length', y='Petal width', data = groups, label = name,
    ↪alpha = 0.3)

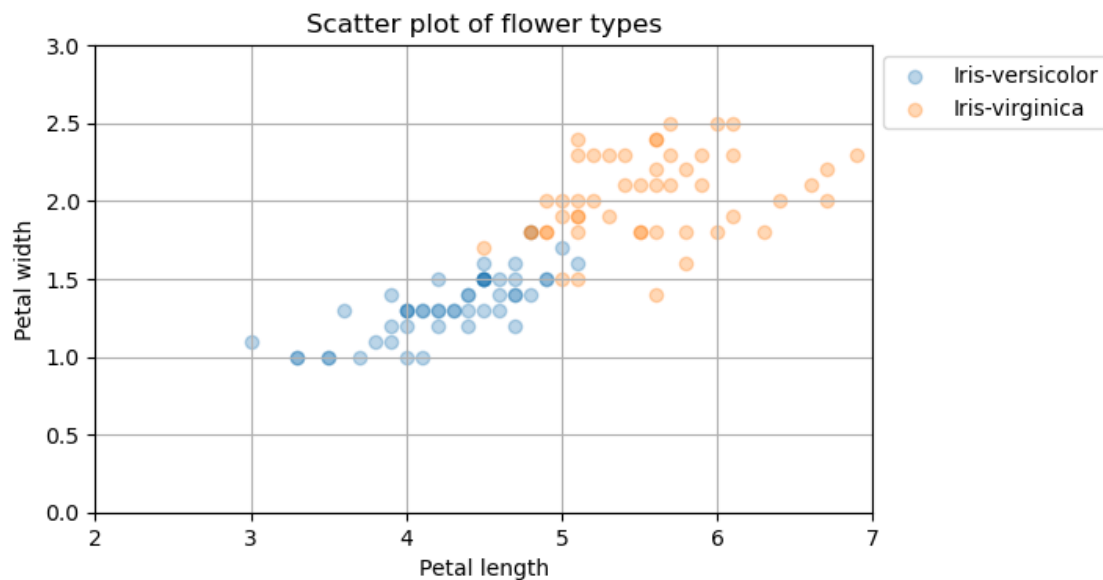
```

```

# set axes and labels
ax.axis('scaled')
ax.axis(axlim)
ax.legend(loc = 'upper left', bbox_to_anchor = (1, 1))
plt.xlabel('Petal length')
plt.ylabel('Petal width')
plt.grid(True)
plt.title('Scatter plot of flower types')

plt.show()

```



From the scatter plot above, you may notice that one species of iris flowers has a stronger correlation between petal width. You may also note that the mean values and spreads (variances) are different.

Part g: Compute the mean vectors, covariance matrices and correlation coefficient matrices of petal length and width for each species.

```

[30]: # Part g mean vectors, covariance matrices, correlation matrices by species

cols = ["Petal length", "Petal width"]
groups = data.groupby("Class")

mean_by_species = {}
cov_by_species = {}
corr_by_species = {}

for sp, g in groups:

```

```

X = g[cols].dropna()

# mean vector (2,)
mean_by_species[sp] = X.mean()

# covariance matrix (2 \times 2), sample covariance (denominator n-1)
cov_by_species[sp] = X.cov()

# correlation coefficient matrix (2 \times 2), pearson correlation
corr_by_species[sp] = X.corr(method = "pearson")

# Pretty print results
for sp in mean_by_species:
    print("=" * 70)
    print(f"Species: {sp}")
    print("\nMean vector [E(L), E(W)]:")
    print(mean_by_species[sp])
    print("\nCovariance matrix:")
    print(cov_by_species[sp])
    print("\nCorrelation coefficient matrix:")
    print(corr_by_species[sp])
    print()

# Covariance: directly print corr(L,W) per species
print("=" * 70)
print("Corr(Petal length, Petal width) by species:")
for sp in corr_by_species:
    r = corr_by_species[sp].loc["Petal length", "Petal width"]
    print(f"{sp}: {r:.6f}")

```

=====

Species: Iris-versicolor

Mean vector [E(L), E(W)]:

Petal length 4.260

Petal width 1.326

dtype: float64

Covariance matrix:

	Petal length	Petal width
Petal length	0.220816	0.073102
Petal width	0.073102	0.039106

Correlation coefficient matrix:

	Petal length	Petal width
Petal length	1.000000	0.786668
Petal width	0.786668	1.000000

```
=====
Species: Iris-virginica
```

```
Mean vector [E(L), E(W)]:
```

```
Petal length    5.552
```

```
Petal width     2.026
```

```
dtype: float64
```

```
Covariance matrix:
```

```
          Petal length  Petal width
Petal length    0.304588    0.048824
Petal width     0.048824    0.075433
```

```
Correlation coefficient matrix:
```

```
          Petal length  Petal width
Petal length    1.000000    0.322108
Petal width     0.322108    1.000000
```

```
=====
Corr(Petal length, Petal width) by species:
```

```
Iris-versicolor: 0.786668
```

```
Iris-virginica: 0.322108
```

Part h: Based on the results in **Part g** and the scatter plots for each species, answer the following questions:

- (1) On average, which species of iris flowers has shorter petals?
- (2) Which species of iris flowers shows greater variation in petal length?
- (3) Which species of iris flowers has a larger correlation coefficient between the petal length and width?

Part h answer:

- (1) *Versicolor* has shorter petals in average (its means petal length is smaller than virginica).
- (2) *Verginica* shows rrgeater variation in petal length (larger standards deviation / variance in the covariance matrix).
- (3) *Versicolor* has a larger correlation between petal length and width (higher Pearson Correlation).

Section 3: Pattern classification with petal length and width

A simple classifier classifies flowers as virginica if the petal length L is greater threshold T , and as versicolor otherwise.

The probability that this classifier makes an error is given by

$$P[\text{vers and } L > T] + P[\text{virg and } L \leq T] = P[L > T|\text{vers}]P[\text{vers}] + P[L \leq T|\text{virg}]P[\text{virg}]$$

Below, we plot the probability that this classifier makes an error for t from 2 to 7 under the following assumptions:

1. The conditional probability density functions of petal length given species, $f_{L|C}(l|c)$, are given by Gaussian distributions
2. The prior probabilities of the two species are equal, i.e. $P[\text{vers}] = P[\text{virg}] = 0.5$

We calculate the probabilities that $L > T$ and $L \leq T$ using the survival functions and the cumulative distribution functions.

The plot shows that there is an optimal threshold at $T = 1.6$. The corresponding probability of making an error is 0.07.

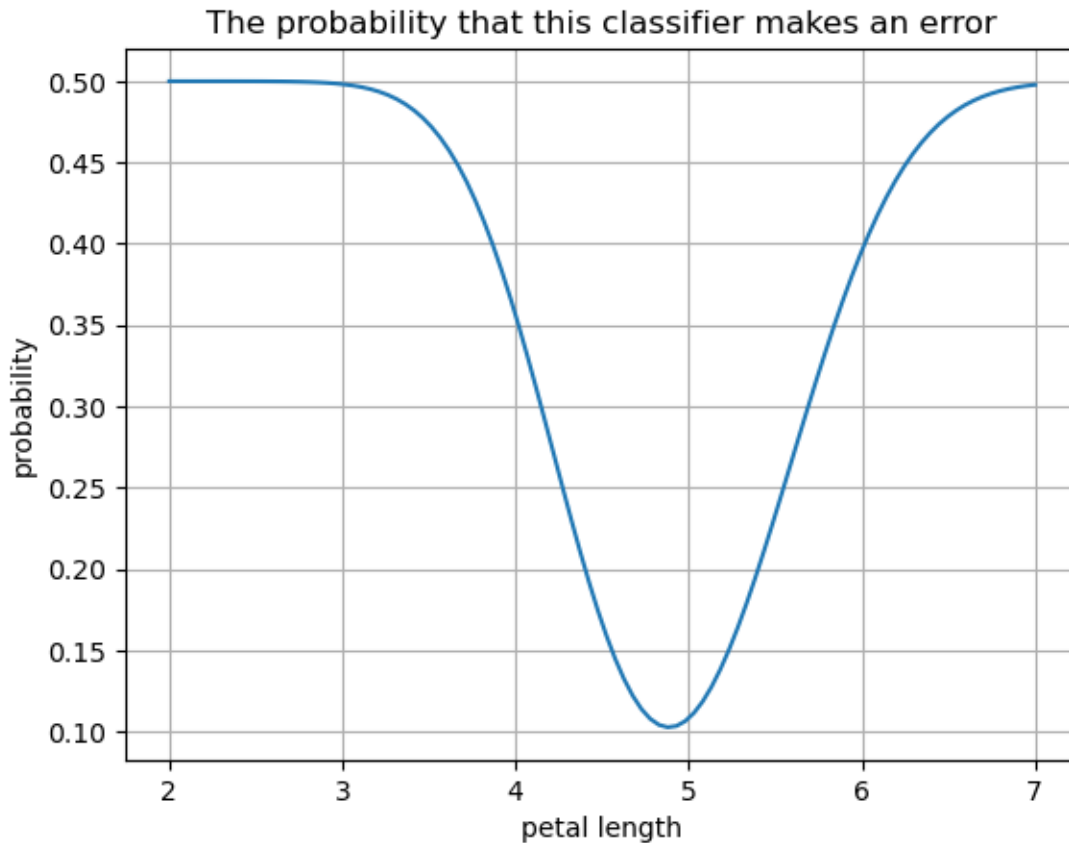
```
[31]: groups = data.groupby('Class')
m_pw_by_species = groups['Petal length'].mean()
s_pw_by_species = groups['Petal length'].std()

t = np.linspace(2, 7, 100)

p_pw_e_versicolor = scipy.stats.norm.sf(t, loc =
    ↪m_pw_by_species['Iris-versicolor'], scale =
    ↪s_pw_by_species['Iris-versicolor'])
p_pw_e_virginica = scipy.stats.norm.cdf(t, loc =
    ↪m_pw_by_species['Iris-virginica'], scale = s_pw_by_species['Iris-virginica'])
p_pw_e = p_pw_e_versicolor*0.5 + p_pw_e_virginica*0.5

fig = plt.figure()
plt.plot(t, p_pw_e)
plt.title('The probability that this classifier makes an error')
plt.xlabel('petal length')
plt.ylabel('probability')
plt.grid()
plt.show()

print(f'The minnum probability that this classifier makes an error is
    ↪{min(p_pw_e)}')
print(f'The optimum threshold is {t[np.argmin(p_pw_e)]}')
```



The minmum probability that this classifier makes an error is

0.10260808519208581

The optimum threshold is 4.878787878787879

Intuitively, we should be able to get better classificaiton peformance if we use more information.

For example, we can consider the difference, $D = 7 * W - L$, where W is the petal width and L is the petal length.

Part i: Consider a classifier that classifies flowers as virginica if $D = 7 * W - L$ is greater threshold T .

Plot the probability that this classifier makes an error for T from 0 to 15 under the assumption that the conditional joint probability density functions of petal width W and petal length L given the species C , $f_{W,L|C}(x,y|c)$ for $c \in \{\text{versicolor}, \text{virginica}\}$, are given by Gaussian distributions with mean vectors and covariance matrices computed in **Part g**.

Find the optimal vaue of T and the correspodng probability that this classifier makes an error.

Is the probability of making an error less than that when we only looked at one feature alone?

```
[32]: # part i
      from scipy.stats import norm
```

```

# 0) setup & extract columns
W_col = "Petal width"
L_col = "Petal length"
C_col = "Class"

sp_vs = "Iris-versicolor"
sp_v = "Iris-virginica"

# Keep only the two classes used in this part
df = data.loc[data[C_col].isin([sp_vs, sp_v]), [C_col, W_col, L_col]].dropna()

# Priors (empirical)
n_vs = (df[C_col] == sp_vs).sum()
n_v = (df[C_col] == sp_v).sum()
pi_vs = n_vs / (n_vs + n_v)
pi_v = n_v / (n_vs + n_v)

# 1) Estimate Gaussian params for joint (W,L) / class
# mu_c (2,), Sigma_c (2 \times 2)

def mean_cov_2d(subdf):
    X = subdf[[W_col, L_col]].to_numpy()    # columns order: [W, L]
    mu = X.mean(axis = 0)                  # shape (2,)
    Sigma = np.cov(X.T, ddof = 1)
    return mu, Sigma

mu_vs, Sigma_vs = mean_cov_2d(df[df[C_col] == sp_vs])
mu_v, Sigma_v = mean_cov_2d(df[df[C_col] == sp_v])

# 2)  $D = 7W - L$  is a linear transform of (W, L)
# If  $(W, L)/c \sim N(\mu_c, \Sigma_c)$  then  $D/c \sim N(mD_c, sD_c^2)$ 
a = np.array([7.0, -1.0])                #  $D = a^T [W, L]$ 

mD_vs = float(a @ mu_vs)
vD_vs = float(a @ Sigma_vs @ a)
sD_vs = float(np.sqrt(vD_vs))

mD_v = float(a @ mu_v)
vD_v = float(a @ Sigma_v @ a)
sD_v = float(np.sqrt(vD_v))

# 3) Error probability as a functional threshold
# rule: predict virginica if  $D > T$ , else versicolor
#  $P_{err}(T) = \pi_{vs} * P(D > T|vs) + \pi_v * P(D \leq T|v)$ 
T_vals = np.linspace(0.0, 15.0, 1501)    # step = 0.01

```

```

# survival function =  $P(D > T)$ 
P_D_gt_T_vs = norm.sf(T_vals, loc = mD_vs, scale=sD_vs)
# cdf =  $P(D \leq T)$ 
P_D_le_T_v = norm.cdf(T_vals, loc=mD_v, scale = sD_v)

P_err = pi_vs * P_D_gt_T_vs + pi_v * P_D_le_T_v

# optimal threshold on this grid
idx = int(np.argmin(P_err))
T_star = float(T_vals[idx])
P_star = float(P_err[idx])

# 4) Plot error vs T
plt.figure(figsize = (7.5, 4.5))
plt.plot(T_vals, P_err, linewidth = 2)
plt.axvline(T_star, linestyle = "--")
plt.xlabel("Threshold T")
plt.ylabel("Probability of error")
plt.title("Part i: Classifier error vs threshold (Gaussian class-conditionals)")
plt.grid(alpha=0.3)
plt.show()

print(f"Priors: P(versicolor) = {pi_vs:.4f}, P(virginica) = {pi_v:.4f}")
print("D|class parameter")
print(f"    versicolor: mean = {mD_vs:.4f}, std{sD_vs:.4f}")
print(f"    virginica:   mean = {mD_v:.4f},   std{sD_v:.4f}")
print(f"\nOptional threshold (grid): T*={T_star:.4f}")
print(f"Minimum error probability: P_err(T*) = {P_star:.6f}")

# 5) Compare with "one feature alone" (petal length L only)
# rule: predict virginica if  $L > t$  else versicolor (1D Gaussian model)

L_vs = df.loc[df[C_col] == sp_vs, L_col].to_numpy()
L_v = df.loc[df[C_col] == sp_v, L_col].to_numpy()

mL_vs, sL_vs = float(L_vs.mean()), float(L_vs.std(ddof = 1))
mL_v, sL_v = float(L_v.mean()), float(L_v.std(ddof = 1))

t_vals = np.linspace(2.0, 7.0, 2001) # reasonable L range
P_L_gt_t_vs = norm.sf(t_vals, loc = mL_vs, scale = sL_vs) # predict v
↳ incorrectly
P_L_le_t_v = norm.cdf(t_vals, loc = mL_v, scale = sL_v) # predict vs
↳ incorrectly

P_err_1d = pi_vs * P_L_gt_t_vs + pi_v * P_L_le_t_v

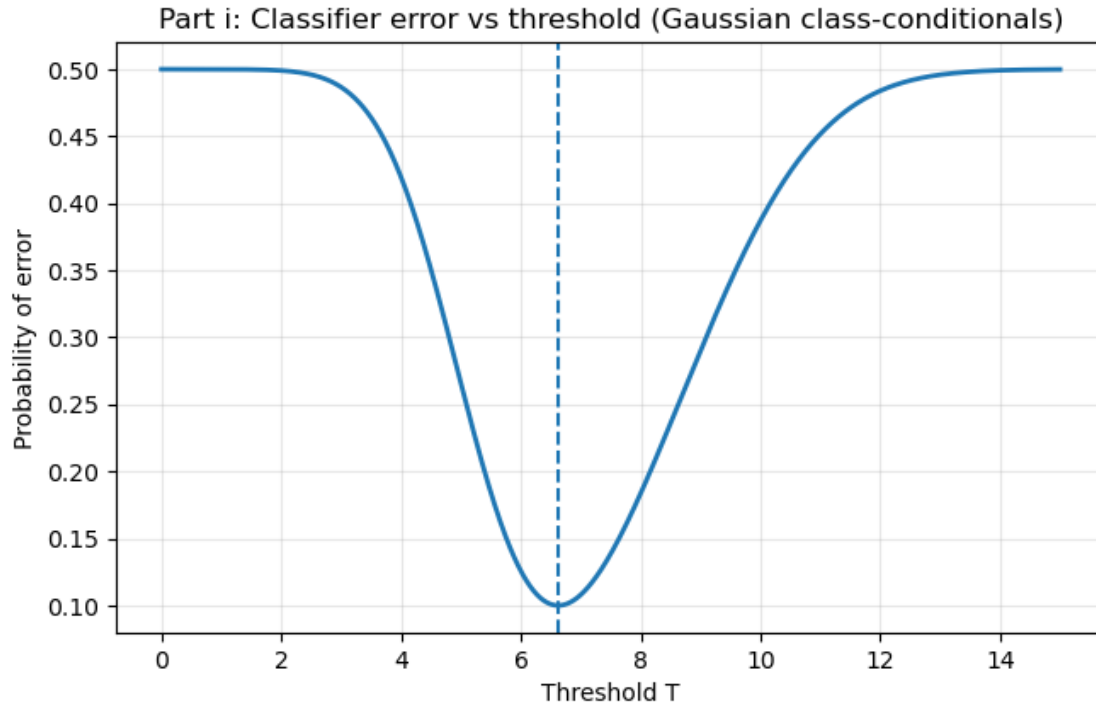
```

```

j = int(np.argmin(P_err_1d))
t_star = float(t_vals[j])
P_star_1d = float(P_err_1d[j])

print("\n[1d baseline using L only]")
print(f"Optimal length threshold t*={t_star:.4f}")
print(f"Minimal error (L only) = {P_star:.4f}")
print(f"Improvement (1D -2D) = {P_star_1d - P_star:.6f}")

```



Priors: $P(\text{versicolor}) = 0.5000$, $P(\text{virginica}) = 0.5000$

D|class parameter

versicolor: mean = 5.0220, std1.0553

virginica: mean = 8.6300, std1.0553

Optional threshold (grid): $T^*=6.6200$

Minimum error probability: $P_{\text{err}}(T^*) = 0.099930$

[1d baseline using L only]

Optimal length threshold $t^*=4.8875$

Minimal error (L only) = 0.0999

Improvement (1D -2D) = 0.002654

Part i answer: (1) Using the Gaussian classconditional model for $(W, L)|C$ and the rule “virginica

if $D = 7W - L > T$, the error probability is

$$p_e(T) = P(C = VC)P(D > T|VC) + P(C = V)P(D \leq T|V)$$

where $D|C = c$ is normal because it is a linear function of a multivariate normal vector. From the error curve $T \in [0, 15]$, the optimal threshold is $T^* = 6.62000$ and the minimum error probability is $P_e(T^*) = 0.09993$

(2) Compared to a one-feature classifier using only petal length L (Gaussian 1D model), the minimum error using $D = 7W - L$ is lower. This indicates that combining both features (width and length) provides better separability than using a single feature along.